TINY Scanner 18 Manual Test Cases

Preparation and common checks

Build scanner.exe once before testing:

```
g++ -std=c++11 -Iinclude src/main.cpp src/scanner.cpp src/tokens.cpp -o scanner.exe
```

For each ordinary case, replace the contents of my-input.txt with the displayed input and save it as UTF-8 without a BOM. Do not add surrounding quotation marks. Use actual line breaks where shown.

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

Read $LASTEXITCODE immediately after the scanner exits. Compare every output line, its order, the summary token count, and warning count and line numbers. Warnings appear in the terminal and must not appear in output.txt. Successful scans should have no stderr output and exit with code 0.

Expected pairs use the compact notation value/TYPE. Each pair means one exact output line written as value , TYPE. For example, x/IDENTIFIER means x , IDENTIFIER; //DIV means / , DIV. An empty token file contains no token lines.

Special cases give their own setup commands. PowerShell WriteAllText is used where an empty file or a file with no trailing newline must be reproduced exactly. Backtick escape sequences are interpreted by PowerShell.

Case 01 All assignment token types

Input

```
; if then end repeat until abc := read write < = + - * / ( ) 12 289
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected ;/SEMICOLON; if/IF; then/THEN; end/END; repeat/REPEAT; until/UNTIL; abc/IDENTIFIER; :=/ASSIGN; read/READ; write/WRITE; </LESSTHAN; =/EQUAL; +/PLUS; -/MINUS; */MULT; //DIV; (/OPENBRACKET;)/CLOSEDBRACKET; 12/NUMBER; 289/NUMBER No warnings. Exit 0.

result PASS. Exact output and recorded checks match. 20 token(s), 0 warning(s), exit 0.

output.txt

```
; , SEMICOLON
if , IF
then , THEN
end , END
repeat , REPEAT
until , UNTIL
abc , IDENTIFIER
:= , ASSIGN
read , READ
write , WRITE
< , LESSTHAN
= , EQUAL
+ , PLUS
- , MINUS
* , MULT
/ , DIV
( , OPENBRACKET
) , CLOSEDBRACKET
12 , NUMBER
289 , NUMBER

```

Case 02 Multiline factorial program

Input

```
{ Sample program in TINY language - computes factorial }
read x; {input an integer}
if 0 < x then
  fact := 1;
  repeat
    fact := fact * x;
    x := x - 1
  until x = 0;
  write fact
end
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected read/READ; x/IDENTIFIER; ;/SEMICOLON; if/IF; 0/NUMBER; </LESSTHAN; x/IDENTIFIER; then/THEN; fact/IDENTIFIER; :=/ASSIGN; 1/NUMBER; ;/SEMICOLON; repeat/REPEAT; fact/IDENTIFIER; :=/ASSIGN; fact/IDENTIFIER; */MULT; x/IDENTIFIER; ;/SEMICOLON; x/IDENTIFIER; :=/ASSIGN; x/IDENTIFIER; -/MINUS; 1/NUMBER; until/UNTIL; x/IDENTIFIER; =/EQUAL; 0/NUMBER; ;/SEMICOLON; write/WRITE; fact/IDENTIFIER; end/END No warnings. Exit 0.

result PASS. Exact output and recorded checks match. 32 token(s), 0 warning(s), exit 0.

output.txt

```
read , READ
x , IDENTIFIER
; , SEMICOLON
if , IF
0 , NUMBER
< , LESSTHAN
x , IDENTIFIER
then , THEN
fact , IDENTIFIER
:= , ASSIGN
1 , NUMBER
; , SEMICOLON
repeat , REPEAT
fact , IDENTIFIER
:= , ASSIGN
fact , IDENTIFIER
```

output.txt

```
* , MULT
x , IDENTIFIER
; , SEMICOLON
x , IDENTIFIER
:= , ASSIGN
x , IDENTIFIER
- , MINUS
1 , NUMBER
until , UNTIL
x , IDENTIFIER
= , EQUAL
0 , NUMBER
; , SEMICOLON
write , WRITE
fact , IDENTIFIER
end , END
```

Case 03 Adjacent tokens without spaces

Input

```
x:=(b+12)/289;
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected x/IDENTIFIER; :=/ASSIGN; (/OPENBRACKET; b/IDENTIFIER; +/PLUS; 12/NUMBER;)/CLOSEDBRACKET; //DIV; 289/NUMBER; ;/SEMICOLON No warnings. Exit 0.

result PASS. Exact output and recorded checks match. 10 token(s), 0 warning(s), exit 0.

output.txt

```
1  x , IDENTIFIER
2  := , ASSIGN
3  ( , OPENBRACKET
4  b , IDENTIFIER
5  + , PLUS
6  12 , NUMBER
7  ) , CLOSEDBRACKET
8  / , DIV
9  289 , NUMBER
10 ; , SEMICOLON
11
```

Case 04 Keyword case and word boundaries

Input

```
if If IF thenx readx
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

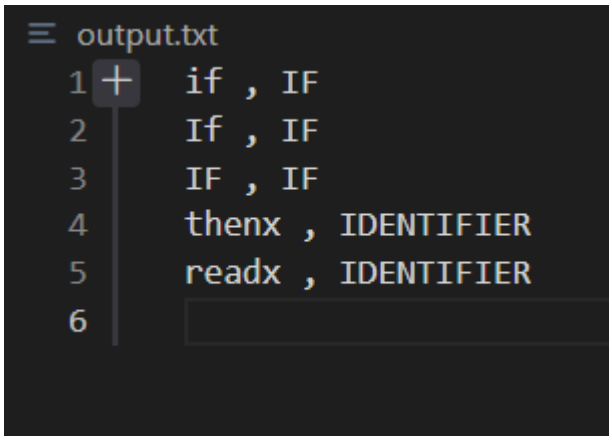
2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected if/IF; If/IDENTIFIER; IF/IDENTIFIER; thenx/IDENTIFIER; readx/IDENTIFIER. No warnings. Exit 0.

result PASS. The revised five-token case was rerun: exact output matches, 5 token(s), 0 warning(s), exit 0.



Case 05 Empty file

Input

```
(Empty file)
```

Steps

1. Create my-input.txt with the exact input using this PowerShell command:

```
[System.IO.File]::WriteAllText("$PWD/my-input.txt", "")
```

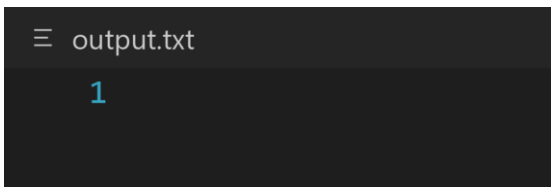
2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected Empty token file. No warnings. Exit 0.

result PASS. Exact output and recorded checks match. 0 token(s), 0 warning(s), exit 0.



Empty file, 0 bytes.

Case 06 Comments only

Input

```
{} { abc @ := 12 }
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

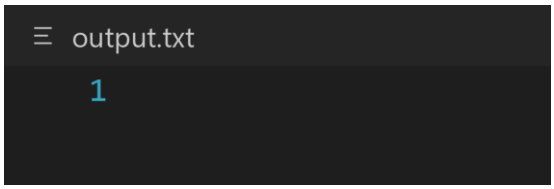
2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected Empty token file. No warnings. Exit 0.

result PASS. Exact output and recorded checks match. 0 token(s), 0 warning(s), exit 0.



Empty file, 0 bytes.

Case 07 Multiline comments and recovery

Input

```
read{a
@ := ignored
}x;
write x
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

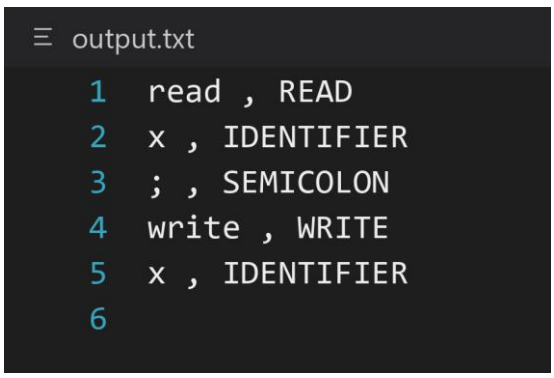
2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected read/READ; x/IDENTIFIER; ;/SEMICOLON; write/WRITE; x/IDENTIFIER No warnings. Exit 0.

result PASS. Exact output and recorded checks match. 5 token(s), 0 warning(s), exit 0.



Case 08 Unknown ASCII characters

Input

```
x @ # $ 4;
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

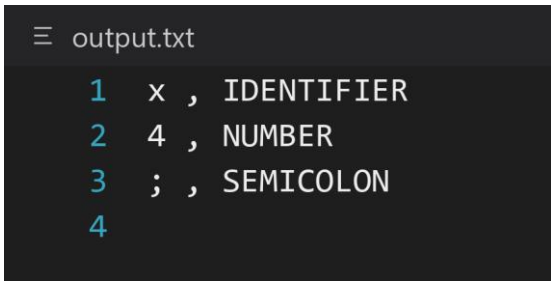
2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected x/IDENTIFIER; 4/NUMBER; ;/SEMICOLON Warnings: Line 1: invalid token '@'; Line 1: invalid token '#'; Line 1: invalid token '$'. Exit 0.

result PASS. Exact output and recorded checks match. 3 token(s), 3 warning(s), exit 0.



Case 09 Lone colon then valid assignment

Input

```
x : 3; y:=4
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

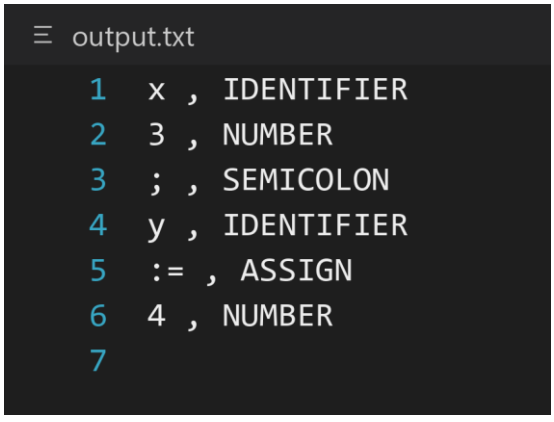
2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected x/IDENTIFIER; 3/NUMBER; ;/SEMICOLON; y/IDENTIFIER; :=/ASSIGN; 4/NUMBER Warnings: Line 1: invalid token ':'. Exit 0.

result PASS. Exact output and recorded checks match. 6 token(s), 1 warning(s), exit 0.



Case 10 Malformed alphanumeric words

Input

```
abc1 12x aar12you write good
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

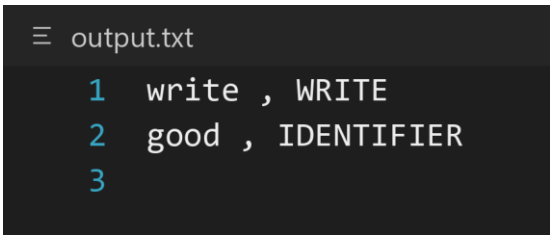
2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected write/WRITE; good/IDENTIFIER Warnings: invalid token 'abc1'; invalid token '12x'; invalid token 'aar12you'. Exit 0.

result PASS. Exact output and recorded checks match. 2 token(s), 3 warning(s), exit 0.



Case 11 Unexpected closing brace

Input

```
read x } write x
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

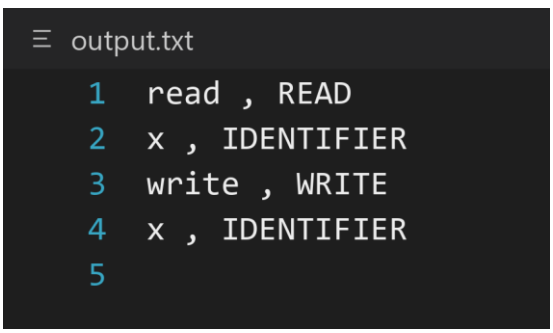
2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected read/READ; x/IDENTIFIER; write/WRITE; x/IDENTIFIER Warnings: Line 1: invalid token '}'. Exit 0.

result PASS. Exact output and recorded checks match. 4 token(s), 1 warning(s), exit 0.



Case 12 Unterminated comment on one line

Input

```
x { never closes
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected x/IDENTIFIER Warnings: Line 1: invalid token '{'. Exit 0.

result PASS. Exact output and recorded checks match. 1 token(s), 1 warning(s), exit 0.

output.txt

```
1  x , IDENTIFIER
2
```

Case 13 Warning line after LF and comments

Input

```
read x
{a
b}
@
write x
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected read/READ; x/IDENTIFIER; write/WRITE; x/IDENTIFIER Warnings: Line 4: invalid token '@'. Exit 0.

result PASS. Exact output and recorded checks match. 4 token(s), 1 warning(s), exit 0.

output.txt

```
1  read , READ
2  x , IDENTIFIER
3  write , WRITE
4  x , IDENTIFIER
5
```

Case 14 Explicit output path

Input

```
read x; write x
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected read/READ; x/IDENTIFIER; ;/SEMICOLON; write/WRITE; x/IDENTIFIER No warnings. Exit 0.

result PASS. Exact output and recorded checks match. 5 token(s), 0 warning(s), exit 0.

output.txt

```
1  read , READ
2  x , IDENTIFIER
3  ; , SEMICOLON
4  write , WRITE
5  x , IDENTIFIER
6
```

Case 15 Missing input file

Input

```
(my-input.txt must be absent)
```

Steps

1. Save any input you need elsewhere, then ensure my-input.txt is absent. Clear any old output.txt before running so it cannot be mistaken for new output.

2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
```

3. Confirm the file error, no success summary, and no newly created output.txt.

Expected Error on stderr: cannot open input file; no success summary and no output file. Exit 1.

result PASS. The expected file error appears on stderr; no success summary; exit 1.

stderr

```
1  Error: cannot open input file 'my-input.txt'
```

Case 16 Nonexistent output directory

Input

```
read x
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

2. Confirm there is no directory named absent under the project root. Run with output.txt inside that missing directory:

```
.\scanner.exe my-input.txt .\absent\output.txt
$LASTEXITCODE
```

3. Verify the error, absence of a success summary, and that absent/output.txt was not created. An old root output.txt is unrelated to this run.

Expected Error on stderr: cannot create output file; no success summary and no output file. Exit 1.

result PASS. The expected file error appears on stderr; no success summary; exit 1.

stderr

```
1  Error: cannot create output file '.\absent\output.txt'
```

Case 17 Last token ends at EOF

Input

```
read xyz
```

Steps

1. Create my-input.txt with the exact input using this PowerShell command:

```
[System.IO.File]::WriteAllText("$PWD/my-input.txt", "read xyz")
```

2. Run the scanner and immediately check its exit code:

```
.\scanner.exe my-input.txt output.txt
$LASTEXITCODE
Get-Content output.txt
```

3. Compare output.txt and the terminal summary and warnings with the expected result below.

Expected read/READ; xyz/IDENTIFIER No warnings. Exit 0.

result PASS. Exact output and recorded checks match. 2 token(s), 0 warning(s), exit 0.

output.txt

```
1  read , READ
2  xyz , IDENTIFIER
3
```

Case 18 Input and output resolve to same file

Input

```
read x;
```

Steps

1. Replace the contents of my-input.txt with the input above and save it.

2. This case deliberately uses my-input.txt for both arguments and can overwrite it. Use only the disposable input above. Run:

```
.\scanner.exe my-input.txt my-input.txt
$LASTEXITCODE
Get-Content my-input.txt
```

3. Check whether the scanner rejects the matching paths or preserves the source. output.txt is not used in this collision check. Recreate my-input.txt before the next case.

Expected Reject the identical paths or preserve the source. Do not replace the input. Tokens themselves would be read/READ, x/IDENTIFIER, ;/SEMICOLON.

result FAIL. Source text read x; is replaced by read , READ / x , IDENTIFIER / ; , SEMICOLON. The process reports success and exits 0. Supplemental data-safety failure, rather than a mismatch with a required token type.

